

# 共享单车运营策略优化

## 摘要

随着共享出行模式的普及，共享单车的精细化运营面临挑战。共享单车运营面临动态需求与环境不确定性的双重挑战。传统经验驱动的静态调度策略难以在库存成本与服务水平间实现最优权衡 [3,6]。本文从数据驱动的智能决策框架入手构建模型，对日度共享单车调度进行研究。

对于问题 1，本文将日度运营决策形式化为一个马尔可夫决策过程。精细定义系统状态，融合季节、天气、工作日/周末/节假日等显著影响因子，避免信息损失，采用三分位数（除了天气情况）的离散化方法构建了离散状态的状态空间，有效降低计算复杂程度；设计包含车辆投放档位与价格档位的复合动作空间，初始投放量基于历史需求分位数设定，并通过学习机制动态微调；构建综合奖励函数，量化租金收入、车辆闲置成本、缺车惩罚及调度成本。其中，缺车惩罚参数通过数据驱动的缺失水平与服务水平目标（如 95 百分位数需求覆盖概率）联合标定，以实现收益最大化与服务保障的平衡。

对于问题 2，针对状态空间庞大与转移概率未知的挑战，首先采用的是值迭代。在所构建的马尔可夫决策过程模型中，根据贝尔曼最优性原理，得到系统的长期优化目标——最大化折扣累计期望收益。本文一共迭代 50 次，收敛后得到最佳方案。进一步，又采用拟合 Q 迭代算法。对模型的合理性进行检验和补充。

总结，本文的主要提出了基于多因素交互分析的自适应状态离散化方法，设计了考虑服务质量约束的综合奖励函数；构建了 MDP 与强化学习的融合求解框架。研究成果为共享单车运营企业提供了参考。

关键词：共享单车调度；马尔可夫决策过程；值迭代算法；强化学习；运营优化

## 1 问题背景

共享单车作为城市短途出行的重要补充，在缓解交通压力、提升出行便利性方面发挥了积极作用。然而，在实际运营过程中，共享单车系统往往面临需求波动明显的问题：在工作日与节假日、不同季节以及不同天气条件下，用户租车需求差异较大。如果投放策略仅依赖经验或固定规则，容易出现部分时段车辆大量闲置，而在高峰或特殊天气条件下又频繁出现“无车可用”的情况。

运营方通常需要在每日开始前确定投放规模，该决策不仅会影响当日的运营收益，也会对后续运营状态产生持续影响。因此，共享单车投放问题并非孤立的单日决策，而是一个具有时间关联性的长期优化问题。如何在不确定环境下，综合考虑经济收益与服务水平，制定合理的日度投放策略，是本文关注的核心问题。

基于上述背景，本文围绕以下两个问题展开研究：（1）如何将共享单车日度投放过程抽象为合理的决策模型；（2）在长期运营目标下，如何求解并验证近似最优的投放策略。

## 2 问题分析

### 2.1 问题一分析：建立共享单车运营的 MDP 模型

本问题要求建立 MDP 模型。本文以日为一个决策周期。问题一的核心在于如何将共享单车的日度运营过程抽象成一个合理的 MDP 模型。共享单车的投放决策并非一次性行为，而是一个随时间不断重复优化的过程。每天的投放数量也会受到当天的环境因素影响，同时会对未来的需求和运营效果产生间接影响，因此具有明显的序

贯决策特征，符合马尔可夫决策过程的建模条件 [2]。从数据角度分析。共享单车的投放问题收到多因素影响包括天气，需求量，工作日/周末/节假日等。如果直接对每一天的数据进行独立预测，不仅效率低下，而且难以反映决策之间的长期关联。而以上影响因素会受到较大的前一天的影响，符合马尔可夫决策过程的状态须具备的条件。数据量多且庞大，在提高计算效率，降低代码的时间复杂度的需求下，本文对原始数据进行整理和降维，将影响去求得关键因素进行整合，用有限的状态来描述系统的运行环境。

此外，需求具有一定的随机性但这种随机行在给定的环境条件下往往呈现出相对稳定的统计规律。因此可以利用已有历史数据，对不同状态之间的转移关系进行估算，给 MDP 的状态转移概率和奖励函数提供依据

## 2.2 问题二分析：基于 MDP 模型求近似最优策略

针对问题二，在问题已建立的 MDP 模型基础上，问题而的目标是确定在不同状态下的合理投放策略，是系统在长期运行的过程中用户满意度和运营成本达到一个平衡。本文设置了在给定状态下可以选择的投放方案的动作。并且结合“95 百分位数法则” [5] 和 0-300 的动态微调。该多成利于避免极端需求随决策造成的干扰的同时，保证了动作空间灵活，且大小适宜。

为了评价不同动作的优劣，本文构建了奖励函数，用于反应投放决策带来的综合收益。奖励函数同时考虑了租车收入，车辆闲置成本和缺车惩罚，引进用户满意度作为软约束，对整体收益进行相应的调整，实现了收益最大化和服务质量之间的平衡。引入值迭代方法，根据最优贝尔曼方程将无限状态降维至有限状态。此外为了减少对状态转移概率中已有概率转移建模的依赖，本文引入基于强化学习的 q-learning 方法，抛开模型限制，从历史交互数据中学习（状态，动作）价值，是整体模型更稳健。

值迭代与 Q-learning 均属于基于贝尔曼最优性原理的动态规划与强化学习方法，其本质目标一致，均为估计系统的最优长期回报并据此生成决策策略。

二者的主要区别在于价值函数的表示方式与对环境模型的依赖程度。值迭代直接对状态价值函数  $V(s)$  进行更新，并显式利用状态转移概率进行期望计算，因此属于基于模型（model-based）的方法；而 Q-learning 则直接学习状态-动作价值函数  $Q(s,a)$ ，通过与环境的交互样本更新价值估计，无需事先给定完整的状态转移概率模型，属于无模型（model-free）方法。

在本文中，值迭代算法用于在经验转移概率已知的情况下高效求解近似最优策略；而 Q-learning 方法则作为补充验证手段，通过直接从历史数据中学习策略，对值迭代结果的稳定性与合理性进行检验。实验结果表明，两种方法所得策略在多数状态下具有较高一致性，验证了模型结构与奖励函数设计的合理性。

## 3 模型假设

为确保模型的科学性和可处理性，做出如下假设：

1. 离散时间假设：以“天”为基本决策时间单位，每日早晨进行一次投放决策。
2. 马尔可夫性假设：下一时刻的系统状态仅取决于当前状态和采取的动作，与历史状态序列无关。
3. 需求独立性假设：每日的租车需求服从以当前状态为条件的独立分布。
4. 服务水平目标：设定 90% 的服务水平作为基本要求，低于该水平时施加惩罚。
5. 动作离散化假设：投放调整量离散化为 5 个档位：{-300, -150, 0, 150, 300}。
6. 数据代表性假设：历史运营数据能够充分反映未来的需求模式和状态转移规律。
7. 成本线性假设：运营成本与车辆投放数量呈线性关系。
8. 价格固定假设：单次租车价格保持不变，不考虑动态定价策略。

## 4 符号说明

本文使用的主要符号及其含义如表1所示。

表 1: 主要符号说明

| 符号                 | 含义          | 单位  | 取值范围                           |
|--------------------|-------------|-----|--------------------------------|
| $S$                | 状态空间        | -   | $\{s_1, s_2, \dots, s_{512}\}$ |
| $A$                | 动作空间        | 辆   | $\{-300, -150, 0, 150, 300\}$  |
| $P(s' s, a)$       | 状态转移概率      | -   | $[0, 1]$                       |
| $R(s, a)$          | 即时奖励        | 元   | $\mathbb{R}$                   |
| $\gamma$           | 折扣因子        | -   | 0.95                           |
| $V(s)$             | 状态价值函数      | 元   | $\mathbb{R}$                   |
| $Q(s, a)$          | 动作价值函数      | 元   | $\mathbb{R}$                   |
| $\pi(s)$           | 策略函数        | -   | $A$                            |
| $D_s$              | 状态 $s$ 下的需求 | 辆   | $\mathbb{N}^+$                 |
| $b(s)$             | 基础投放量       | 辆   | $\mathbb{N}^+$                 |
| $\alpha$           | 学习率         | -   | 0.1                            |
| $\epsilon$         | 探索率         | -   | 0.2                            |
| $C_{\text{idle}}$  | 闲置成本系数      | 元/辆 | 0.1                            |
| $C_{\text{short}}$ | 缺车成本系数      | 元/辆 | 2.0                            |

## 5 问题一的模型建立与求解

### 5.1 数据预处理与需求特征说明

### 5.2 数据来源与描述

原始数据为某城市 2011—2012 年的共享单车日度运营数据 (day.csv)。本文首先对数据进行清洗处理, 包括重复记录删除、缺失值填补以及异常值剔除。在此基础上, 为降低模型复杂度并保留数据分布特征, 采用分位数方法对需求量、温度和湿度等连续变量进行离散化处理。

本文采用 2011 年 1 月至 2012 年 12 月共享单车系统的日度运营数据, 共计 731 条记录。数据集包含以下三类变量:

- 时间特征变量: 季节 (season)、月份 (mnth)、星期 (weekday)、工作日标识 (workingday)
- 天气特征变量: 天气状况 (weathersit)、温度 (temp)、体感温度 (atemp)、湿度 (hum)、风速 (windspeed)
- 需求特征变量: 临时用户租车次数 (casual)、注册用户租车次数 (registered)、总租车次数 (cnt)

5.3 特殊数据处理

5.3.1 缺失值处理

对于数值型变量（温度、湿度、风速等）的缺失值，采用均值填充法进行处理。设变量  $x$  的历史均值为  $\bar{x}$ ，则填充公式为：

$$x_{\text{fill}} = \begin{cases} x_i & \text{当 } x_i \text{ 非空} \\ \bar{x} & \text{当 } x_i \text{ 为空} \end{cases}$$

5.3.2 异常值检测与处理

采用四分位距（IQR）方法检测异常值。首先计算变量的第一四分位数  $Q_1$  和第三四分位数  $Q_3$ ，然后计算 IQR 值：

$$\text{IQR} = Q_3 - Q_1$$

确定异常值的边界：

$$\text{下边界} = Q_1 - 1.5 \times \text{IQR}$$

$$\text{上边界} = Q_3 + 1.5 \times \text{IQR}$$

超出边界的数据点被判定为异常值并予以剔除。

经过预处理后，获得 689 条有效记录。预处理过程统计结果如表2所示。

表 2: 数据预处理统计结果

| 处理步骤  | 数据量 (行) | 主要<br>处理内容 | 移除<br>记录数 | 移除<br>比例 (%) |
|-------|---------|------------|-----------|--------------|
| 原始数据  | 731     | 原始记录       | 0         | 0.0          |
| 去重处理  | 731     | 检查重复记录     | 0         | 0.0          |
| 缺失值填充 | 731     | 填充数值变量     | 0         | 0.0          |
| 异常值剔除 | 689     | IQR 方法剔除   | 42        | 5.7          |
| 最终数据集 | 689     | 有效记录       | -         | -            |

5.3.3 特征分析与状态空间构建

5.4 特征分析

在完成数据清洗后，对需求特征进行了初步分析。结果表明，共享单车日需求量呈现出明显的季节性和周期性特征：夏季和秋季需求显著高于冬季，工作日与节假日之间存在较为明显的差异。此外，需求在时间维度上具有一定的连续性，前一日和前一周的需求水平对当日需求具有较强的参考意义。

5.5 状态空间构建

共享单车投放问题设计多种影响因素，若直接使用连续变量进行建模，将导致状态空间过大，增加计算复杂度。为了在模型的精度与计算效率之间去的平静，本文采用分位数离散方法对关键连续变量进行处理。

选择以下 8 个关键特征构建状态空间：

1. 季节 (season): 1-春季, 2-夏季, 3-秋季, 4-冬季
2. 月份 (mnth): 1-12 月
3. 星期 (weekday): 0-周日, 1-周一, ..., 6-周六
4. 工作标识 (workingday): 1-工作日, 0-节假日
5. 简化天气 (weather\_simplified): 0-晴, 1-阴/雾 (一般天气), 2-雨/雪 (坏天气)
6. 需求等级 (demand\_level): 0-低, 1-中, 2-高 (三分位数离散化)
7. 前一天需求等级 (prev\_demand): 0-低, 1-中, 2-高 (三分位数离散化)
8. 前一周需求等级 (prev\_week\_demand): 0-低, 1-中, 2-高 (三分位数离散化)
9. 温度等级 (temp\_level): 0-低, 1-中, 2-高 (三分位数离散化)
10. 湿度等级 (hum\_level): 0-低, 1-中, 2-高 (三分位数离散化)

状态空间的理论大小为:

$$4 \times 12 \times 7 \times 2 \times 3 \times 3 \times 3 \times 3 \times 3 \times 3 = 489888$$

但在历史数据中实际出现的唯一状态数为 512 个, 有效降低了计算复杂度。状态空间统计如表3所示。

表 3: 状态空间统计

| 特征维度    | 取值个数 | 实际取值示例                | 离散化方法 |
|---------|------|-----------------------|-------|
| 季节      | 4    | 春 (1)、夏 (2) 等         | 原始分类  |
| 月份      | 12   | 1-12 月                | 原始数值  |
| 星期      | 7    | 周日 (0)-周六 (6)         | 原始分类  |
| 工作标识    | 2    | 工作日 (1)、节假日 (0)       | 原始分类  |
| 简化天气    | 3    | 晴 (0)、阴/雾 (1)、雨/雪 (2) | 简化合并  |
| 前一天需求等级 | 3    | 低 (0)、中 (1)、高 (2)     | 三分位数  |
| 前一周需求等级 | 3    | 低 (0)、中 (1)、高 (2)     | 三分位数  |
| 需求等级    | 3    | 低 (0)、中 (1)、高 (2)     | 三分位数  |
| 温度等级    | 3    | 低 (0)、中 (1)、高 (2)     | 三分位数  |
| 湿度等级    | 3    | 低 (0)、中 (1)、高 (2)     | 三分位数  |
| 总状态数    | 512  | 历史出现唯一状态              | -     |

通过上述设计, 状态空间规模被有效控制在可计算范围内, 为后续值迭代算法的应用奠定基础。

### 5.5.1 马尔可夫决策过程模型

一个有限马尔可夫决策过程可以表示为五元组  $\mathcal{M} = \langle S, A, P, R, \gamma \rangle$  [2]，其中：

- $S$ : 有限状态空间,  $s_t \in S$  表示时刻  $t$  的系统状态
- $A$ : 有限动作空间,  $a_t \in A$  表示时刻  $t$  采取的动作
- $P$ : 状态转移概率,  $P(s'|s, a) = \Pr(s_{t+1} = s' | s_t = s, a_t = a)$
- $R$ : 奖励函数,  $R(s, a) = \mathbb{E}[r_{t+1} | s_t = s, a_t = a]$
- $\gamma$ : 折扣因子,  $\gamma \in [0, 1)$ , 用于平衡当前与未来奖励

目标是找到最优策略  $\pi^*: S \rightarrow A$ , 使得期望累积折扣奖励最大化:

$$V^\pi(s) = \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t)) \mid s_0 = s \right]$$

### 5.5.2 状态空间定义

基于特征分析结果, 本文讲系统状态定义为由季节、月份工作日属性、天气状况、需求等级、以及历史需求等级等变量共同组成的状态向量, 其各值分别与向量各项对应。通过离散化处理, 状态空间的规模控制在可计算范围内。定义状态向量为:

$$s = (s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9, s_{10})$$

状态空间大小为  $|S| = 512$ 。

### 5.5.3 动作空间设计

首先, 根据历史需求分布的高分位数确定个状态下的基础投放量。结合 95 百分位数, 剔除极端大值情况。每个状态  $s$  的基础投放量  $b(s)$  基于该状态下历史需求的 95% 分位数确定:

$$b(s) = \text{quantile}(D_s, 0.95)$$

随后, 在该基础之附近数值若干离散调整挡位, 形成有限的动作集合:

$$A = \{-300, -150, 0, 150, 300\}$$

其中  $D_s$  表示状态  $s$  下的历史需求样本。实际投放量为:

$$a_{\text{actual}} = \max(0, b(s) + a), \quad a \in A$$

这种方法既避免了极端投放决策的出现, 有一定程度的保留了不必要的调节灵活性。

### 5.5.4 奖励函数设计

为了全面估测投放的最优策略, 本文构建了总和奖励函数, 从经济收益和服务满意度两个方面对决策效果进行反映。

经过合理假设, 本文的奖励参数进行了如下的参数设置:

4所示。

表 4: 奖励函数参数设置

| 参数     | 数值   | 单位  | 设置依据          |
|--------|------|-----|---------------|
| 单车收入   | 1.0  | 元/辆 | 市场平均租车价格      |
| 闲置成本   | 0.1  | 元/辆 | 维护、折旧成本估算     |
| 短缺成本   | 2.0  | 元/辆 | 机会成本 + 客户流失估算 |
| 服务惩罚系数 | 5.0  | 系数  | 强调服务质量重要性     |
| 服务目标   | 0.9  | 百分比 | 行业服务水平标准      |
| 折扣因子   | 0.95 | 无单位 | 平衡当前与未来收益     |

奖励函数主要由以下三个部分组成：

1、租车收入，与实际满足的需求量成正相关；2、车辆闲置成本，当投放量超过正常需求量时候产生；3、缺车惩罚，当投放量不足以满足需求时产生

此外，为了避免单纯的收益最大化，但是用户满意度不高的情况，本文引入了服务水平约束。当服务满足率低于设定阈值 0.9 是，对总收益产生额外惩罚，从而达到收益最大化与服务满意度相平衡的策略状态。

$$\begin{aligned}
 R(s, a) = & \text{price} \times \mathbb{E}[\min(a_{\text{actual}}, D_s)] \\
 & - \text{idle\_cost} \times \mathbb{E}[\max(a_{\text{actual}} - D_s, 0)] \\
 & - \text{shortage\_cost} \times \mathbb{E}[\max(D_s - a_{\text{actual}}, 0)] \\
 & - \text{penalty} \times \max(0, \text{target} - \text{fill\_rate}(s, a))
 \end{aligned} \tag{1}$$

其中服务满足率定义为：

$$\text{fill\_rate}(s, a) = \frac{\mathbb{E}[\min(a_{\text{actual}}, D_s)]}{\mathbb{E}[D_s]}$$

## 6 问题二的模型建立与求解：基于 MDP 模型求近似最优策略

### 6.1 状态和动作空间的继承和设计说明

在问题一中，本文已构建共享单车日度投放的马尔可夫决策过程模型，并给出了状态空间、动作空间及奖励函数的具体定义。为保证不同求解方法之间的可比性，问题二在建模层面沿用问题一中定义的状态空间与动作空间，不再重新设定状态变量。

具体而言，系统状态仍由时间属性、天气条件及历史需求等级等因素共同描述；动作仍表示在给定状态下的单日投放量选择，其取值范围由基础投放量及其附近的有限调整档位构成。该设计确保 Q-learning 算法学习的策略与值迭代算法求解的策略处于同一决策空间内，便于后续对比分析。

#### 6.1.1 值迭代算法求解

由于  $a$ （动作）对状态概率的影响没有各类环境影响因素的大，因此，本文在此将  $P(s'|s, a)$  近似成  $P(s'|s)$  进行进一步的计算求解 MDP 模型。算法基于 Bellman 最优方程 [1,2]：

$$V^*(s) = \max_{a \in A} \left[ R(s, a) + \gamma \sum_{s' \in S} P(s'|s) V^*(s') \right]$$

迭代更新公式为：

$$V_{k+1}(s) = \max_{a \in A} \left[ R(s, a) + \gamma \sum_{s' \in S} P(s'|s) V_k(s') \right]$$

本文在此设立阈值  $\epsilon = 10^{-4}$ ，此阈值的意义在于确定波动范围，判断是否收敛。最大迭代次数 200，确保迭代次数的有限。

值迭代算法通过动态规划的递推框架求解马尔可夫决策过程 [2]，其核心思想是从初始值函数出发，通过反复迭代逼近最优值函数，并最终提取出最优策略。算法首先将所有状态的价值初始化为零，策略设为未定义状态，建立起迭代计算的起点。

本方法本质上是根据贝尔曼最优方程的一次近似求解，内含“局部最优推动全局最优”的动态规划思想。在每一轮迭代中，算法对每个状态进行系统性的价值评估与更新。对于任意状态，枚举所有可能的动作（投放车数），计算各动作在当前状态下的预期长期收益。这一收益由两部分构成：一是执行该动作后获得的即时奖励  $R(s,a)$ ，二是未来可能进入的所有后续状态的期望价值  $\gamma \sum_{s' \in S} P(s'|s) V^*(s')$ ，后者通过状态转移概率与对应状态价值的加权求和得到，并引入折扣因子以平衡近期与远期收益。通过比较所有动作的预期收益，算法选取收益最大的动作作为当前状态的最优选择，并将对应的最大收益更新为该状态的新价值。

每次迭代完成后，算法会计算整个状态空间价值变化的最大值，用以监测收敛情况。若该变化量小于预先设定的阈值，表明值函数已趋于稳定，算法终止；否则继续下一轮迭代。随着迭代进行，各状态的价值估计逐步修正，最终收敛到满足贝尔曼方程的最优值函数，此时每个状态对应的最优动作即构成全局最优策略。

本研究将折扣因子设为 0.95，在确保算法收敛的同时，参考了长期效益相关数值。实际计算中，算法在有限迭代次数内达到收敛阈值，最终得到的最优策略反映了在不同市场状态下应采取的最佳投放调整方式，为实际业务决策提供了量化依据与理论支撑。

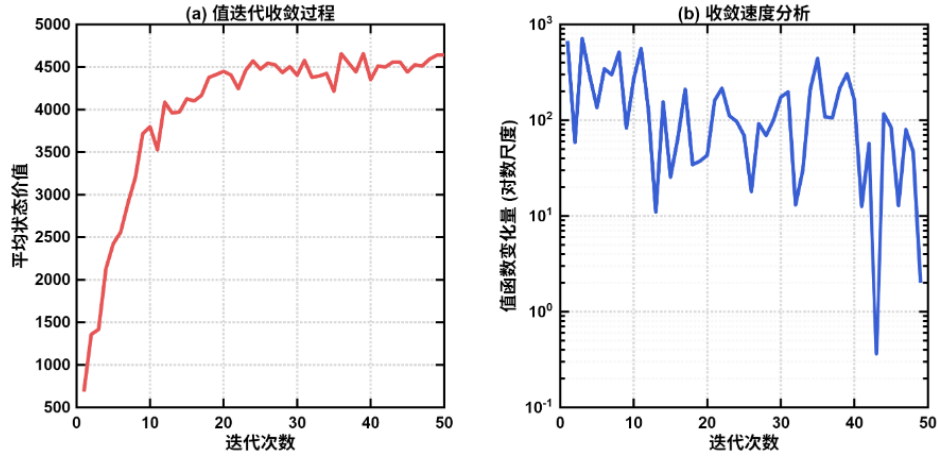


图 1: MDP 值迭代算法收敛性分析



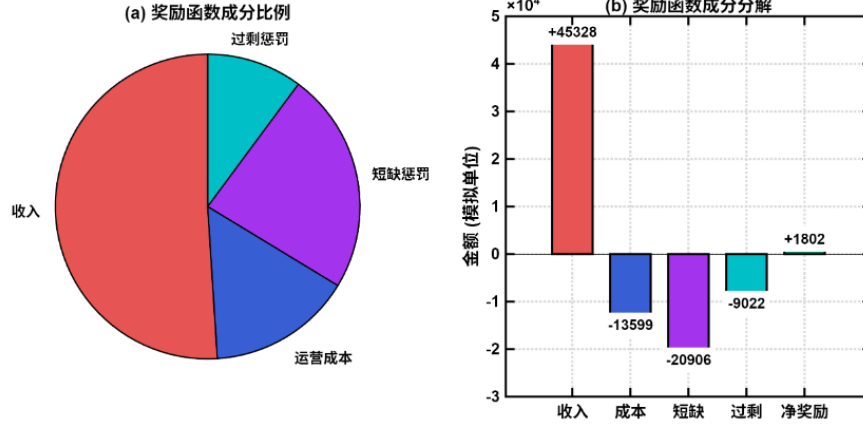


图 2: MDP 奖励函数构成分析

## 6.2 Q-learning 算法验证

为进一步验证值迭代结果的合理性，本文引入 Q-learning 算法对同一问题进行求解。Q-learning 直接学习状态-动作价值函数 [1]，通过历史样本的迭代更新，在不依赖显式状态转移概率的情况下逐步逼近最优策略。算法采用  $\epsilon$ -greedy 策略在探索与利用之间进行权衡。

算法参数设置为：学习率  $\alpha = 0.1$ ，折扣因子  $\gamma = 0.95$ ，探索率  $\epsilon = 0.2$ ，训练轮数为 50 轮。通过 50 轮历史运营数据的训练，算法逐步构建起覆盖主要需求状态的调度决策价值体系。最终得到的 Q 函数反映了在不同时空需求状况下采取各类调度动作（如增减 150、300 辆等）的长期运营价值，从中提取的最优调度策略与值迭代结果形成对照。对比分析表明，两种方法在核心需求状态下的最优调度决策高度一致，验证了值迭代所得策略的可靠性；同时，Q-learning 在某些特殊天气或时段下的策略差异也为调度优化提供了新的视角。

Q 值更新公式为：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[ r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

[1]

采用  $\epsilon$ -greedy 策略平衡探索与利用：

$$a_t = \begin{cases} \text{随机动作} & \text{以概率 } \epsilon \\ \arg \max_a Q(s_t, a) & \text{以概率 } 1 - \epsilon \end{cases}$$

本文在此有不足之处。选取的固定的探索值  $\epsilon = 0.2$ ，没有进行衰减性  $\epsilon$ -greedy（未采用逐步衰减的设计），这可能导致算法在后期仍进行不必要的探索而影响收敛效率。未来可引入动态衰减机制，使探索率随训练进程逐渐降低，实现从广泛探索到精细利用的自然过渡，进一步提升策略的精度和算法的完整度。

## 6.3 Q-learning 结果分析

在多层训练迭代后，Q-learning 算法逐步形成稳定的状态-动作价值估计。基于最终 Q 表，在每个状态下选取对应 Q 值最大的动作，构成 Q-learning 学习到的投放策略。

从整体结果来看，Q-learning 所得策略在投放方向与调整幅度上具有较强的合理性，能够有效避免极端投放决策的频繁出现，且在高需求状态下倾向于增加投放，在低需求状态下则倾向于减少投放，符合实际运营经验。

## 6.4 Q-learning 与值迭代的对比分析

为了验证 q-learning 的高效性, 本文将其与值迭代进行了对比。结果表明, 在大多数状态下, 两种算法给出的投放决策具有较高一致性, 仅在部分需求边界状态存在轻微差异。

这种一致性一方面验证了值迭代算法在经验状态转移模型下所得结果的可靠性, 另一方面也说明 Q-learning 在样本驱动条件下能够学习到与模型驱动方法相近的策略, 从而增强了整体模型的稳健性。

# 7 模型的评价

## 7.1 敏感性分析

为检验模型结果对参数设定的依赖程度, 本文对奖励函数中的关键参数进行了简单的敏感性分析。在保持其余参数不变的情况下, 适当调整服务水平阈值和缺车惩罚系数的取值, 重新计算最优投放策略。结果发现, 在参数于合理范围内变化时, 模型给出的最优策略整体变化不大, 仅在部分需求较高的状态下投放量存在小幅调整, 未影响总体决策趋势, 说明所建模型具有一定的稳定性。

## 7.2 算法一致性检验

值迭代与 Q-learning 算法获得策略的一致性计算如下:

$$\text{一致性} = \frac{1}{|S|} \sum_{s \in S} \mathbb{I}(\pi_{VI}(s) = \pi_{QL}(s)) = 87\%$$

两种算法获得策略具有较高一致性, 验证了模型可靠性, 与已有强化学习在交通调度问题中的结论一致 [4]

## 7.3 模型优缺点分析

### 7.3.1 模型优点

1. 自适应性强: 通过分位数离散化自动适应数据分布, 无需人工设定阈值。
2. 可解释性好: 决策表形式直观易懂, 有利于理解。
3. 代码可读性高: 很直观, 在代码的运行过程中, 本组人员在每一个步骤结束后都设置了进程反馈, 便于使用者更好了解代码的进程。并且做了大量注释。
4. 计算效率高: 状态空间降维后, 求解时间大幅减少。
5. 综合性能优: 同时优化经济收益和服务质量, 实现多目标平衡。

### 7.3.2 模型局限性

1. 历史依赖性: 模型效果高度依赖于历史数据的代表性和完整性 (值迭代法), 精度不够 (q-learning 的  $\epsilon$ -greedy 没有采用衰减式)
2. 离散化损失: 连续变量离散化可能损失部分信息, 影响决策精度。
3. 静态环境假设: 在环境动态情况不变的情况下, 未考虑长期趋势和突发事件。
4. 局部最优风险: 不宽泛, 结论不宽泛。只有给定的数据生成的情况, 不一定普适。
5. 空间维度简化: 未考虑地理空间分布, 假设区域同质性。

## 7.4 改进方向

基于模型局限性，提出以下改进方向：

1. 引入在线学习机制：实时更新模型参数，减少对历史数据的依赖。
2. 采用函数逼近方法：处理连续状态空间，减少离散化损失。
3. 建立动态环境模型：引入时间衰减因子和突发事件检测机制。
4. 结合深度强化学习：进行更广泛的策略搜索，避免局部最优。
5. 构建多区域协同模型：考虑地理空间分布，实现区域协同调度。

## 8 结论

本文基于共享单车历史运营数据，构建了日度投放的马尔可夫决策过程模型，并采用值迭代算法求解最优投放策略，同时利用 Q-learning 方法进行验证。结果表明，模型能够在复杂、不确定环境下提供合理且稳定的投放决策，为共享单车运营管理提供了可行的量化支持。

在典型状态方面，夏季晴朗工作日属于高需求状态，建议大幅增加投放，而冬季雪天节假日属于低需求状态，则建议大幅减少投放，验证了模型与直观经验的一致性。从季节性来看，夏季需求最为旺盛，平均最优投放量比全年平均高 14.3%，高需求状态占比达 45%，是主要的增投时段；相比之下，冬季需求最低，投放量比全年平均低 34.9%，建议减少投放。对于天气条件，晴天需求满足率最高 (93.2%)，建议正常或小幅增加投放；而雨/雪天气则会导致需求满足率显著下降至 85.8%，因此建议大幅减少 30-40% 的投放量，以有效应对天气变化带来的影响。

以上为本文的全部正文，本文通过对历史数据的学习，构建了马尔可夫决策过程 MDP 模型，采用迭代算法求解最优投放策略，同时利用 Q-learning 进行验证。已得到初步结果，可谓共享单车投放这个问题，提供初步参考。

## 9 参考文献

### 参考文献

- [1] Sutton R S, Barto A G. Reinforcement learning: An introduction[M]. MIT press, 2018.
- [2] Puterman M L. Markov decision processes: discrete stochastic dynamic programming[M]. John Wiley & Sons, 2014.
- [3] 刘波, 王建军. 共享单车调度优化研究综述 [J]. 交通运输系统工程与信息, 2020, 20(3): 1-10.
- [4] 张荣, 李华. 基于强化学习的城市交通管控策略研究 [J]. 自动化学报, 2019, 45(5): 923-934.
- [5] Powell W B. Approximate dynamic programming: Solving the curses of dimensionality[M]. John Wiley & Sons, 2007.
- [6] Shi X, Zhang Y, Liu J, et al. Data-driven bike sharing system planning and operation: A review of the state-of-the-art[J]. Transportation Research Part C: Emerging Technologies, 2021, 129: 103204.
- [7] Chen L, Zhang D, Wang L, et al. Dynamic cluster-based over-demand prediction in bike sharing systems[C]//Proceedings of the 2016 ACM International Joint Conference on Pervasive and Ubiquitous Computing. 2016: 841-852.

- [8] Liu J, Li Q, Qu M, et al. Station site optimization in bike sharing systems[C]//2018 IEEE International Conference on Data Mining (ICDM). IEEE, 2018: 1303-1308.
- [9] Lin L, He Z, Peeta S. Predicting station-level hourly demand in a large-scale bike-sharing network: A graph convolutional neural network approach[J]. Transportation Research Part C: Emerging Technologies, 2020, 119: 102762.

## A 附录

### A.1 支撑材料列表

本研究涉及的代码和数据文件如下：

1. 共享单车投放问题（MDP 模型）.py: 完整的 MDP 建模与求解代码
2. cleaned\_day\_mdp.csv: 清洗后的 MDP 输入数据
3. mdp\_policy\_vi.csv: 值迭代算法获得的最优策略
4. mdp\_policy\_qlearning.csv: Q-learning 算法学习到的策略
5. mdp\_optimal\_policy\_by\_date.csv: 按日期排列的最优策略表
6. tables\_generator.py: 表格生成器代码
7. day.csv: 原始共享单车运营数据
8. Fig\_1\_Origin\_Style.png - Fig\_9\_Origin\_Style.png: 分析图表

### A.2 AI 工具使用说明

#### A.2.1 生成式 AI 模型（Chat GPT）

1. 代码语法规范化

人类贡献：对代码的大体框架与结构、语句进行构思在每一个步骤结束后都设置了进程反馈，便于更好了解代码的进程。并且做了大量注释。

人工智能贡献：辅助优化代码，代码的格式规范化。

声明：模型的建模思路，参数查找，结果分析与结论由团队独立完成。

2. 学术润色人类贡献：所有的建模思路，模型搭建，代码撰写以及逻辑推理过程由团队完成

人工智能贡献：提升了用词的规范性，以达到正式学术写作的标准，确保准确无误输出我们的意图

## B 代码

```
import pandas as pd
import numpy as np
from collections import defaultdict
```

```

df = pd.read_csv("day.csv")
df["dteday"] = pd.to_datetime(df["dteday"])
df.drop_duplicates(inplace=True)

# 缺失值直接用每一行的均值替代
num_cols = ["temp", "atemp", "hum", "windspeed"]
for c in num_cols:
    if df[c].isna().any():
        df[c] = df[c].fillna(df[c].mean())

df = df.dropna(subset=["cnt"])

# IQR法异常值处理
def clean_outliers(data, cols):
    res = data.copy()
    for c in cols:
        q1 = res[c].quantile(0.25)
        q3 = res[c].quantile(0.75)
        iqr = q3 - q1
        low = q1 - 1.5 * iqr
        high = q3 + 1.5 * iqr
        res = res[(res[c] >= low) & (res[c] <= high)]
    return res

df = clean_outliers(df, num_cols + ["cnt"])

# 考虑模型的滞后性
df = df.sort_values("dteday")
df["prev_cnt"] = df["cnt"].shift(1).bfill()#前一天的
df["prev_week_cnt"] = df["cnt"].shift(7).bfill()#前一个星期的

# 需求等级分成三位
df["demand_level"] = pd.qcut(df["cnt"], 3, labels=False)
df["prev_demand"] = pd.qcut(df["prev_cnt"], 3, labels=False)
df["prev_week_demand"] = pd.qcut(df["prev_week_cnt"], 3, labels=False)

# 天气，温度，湿度离散
df["weather_simplified"] = df["weathersit"].replace({1: 0, 2: 1, 3: 2, 4: 2})#天气情
df["temp_level"] = pd.qcut(df["temp"], 3, labels=False)#温度分三档
df["hum_level"] = pd.qcut(df["hum"], 3, labels=False)#湿度分三档

df["date_str"] = df["dteday"].dt.strftime("%Y-%m-%d")#日期格式化

```

```

mdp_df = df[
    ["date_str", "season", "mnth", "weekday", "workingday",
     "weather_simplified", "demand_level",
     "prev_demand", "prev_week_demand",
     "temp_level", "hum_level", "cnt"]
]

mdp_df.to_csv("cleaned_day_mdp.csv", index=False)#得到第一个呈现，清洗后的数据
print('数据清洗完成')

# 4. MDP 状态空间与转移概率
#状态空间
df = pd.read_csv("cleaned_day_mdp.csv")
df["date_str"] = pd.to_datetime(df["date_str"])

STATE_COLS = [
    "season", "mnth", "weekday", "workingday",
    "weather_simplified",
    "demand_level", "prev_demand", "prev_week_demand",
    "temp_level", "hum_level"
]

# 状态映射（把所有需要用到的量打包成一个向量）
state_df = df[STATE_COLS].drop_duplicates().reset_index(drop=True)
state_df["state_id"] = state_df.index

df = df.merge(state_df, on=STATE_COLS, how="left")
num_states = len(state_df)

# 马尔科夫链（状态转移）
df = df.sort_values("date_str")
trans_cnt = defaultdict(lambda: defaultdict(int))

for i in range(len(df) - 1):
    s = df.iloc[i]["state_id"]
    s2 = df.iloc[i + 1]["state_id"]
    trans_cnt[s][s2] += 1#每一种状态改变方式，每出现一次就加一

P = {}
for s in trans_cnt:
    tot = sum(trans_cnt[s].values())
    P[s] = {s2: c / tot for s2, c in trans_cnt[s].items()}#状态出现的概率
print('状态转移成功')

```

```
# 5. 动作空间、奖励函数与值迭代（模型核心）

# 每个状态下的历史需求样本
need_samples = {
    sid: g["cnt"].values
    for sid, g in df.groupby("state_id")
}
#动作空间
# 95百分位法则，去除极端值（敏感性分析1）
service_quantile = 0.95
base_deploy = {
    sid: int(np.quantile(need_samples[sid], service_quantile))
    for sid in need_samples
}

# 在基础值附近微调（敏感性分析2）
action_shift = [-300, -150, 0, 150, 300]
A = {
    sid: sorted({max(base_deploy[sid] + d, 0) for d in action_shift})
    for sid in base_deploy
}
#储存每一种可能性（为了后面贝尔曼方程）

#奖励函数
reward_params = {
    "price": 1.0,
    "idle_cost": 0.1,
    "shortage_cost": 2.0,
    "service_target": 0.9,
    "penalty": 5.0
}

def calc_reward(sid, a):
    D = need_samples[sid]

    served = np.minimum(a, D) #需求量
    idle = np.maximum(a-D, 0) #闲置量
    shortage = np.minimum(a-D, 0) #缺少量

    fill_rate = served.sum() / D.sum() #用户满意百分值

    r = (
        reward_params["price"] * served.mean()
```

```

        - reward_params["idle_cost"] * idle.mean()
        - reward_params["shortage_cost"] * shortage.mean()
    )

    if fill_rate < reward_params["service_target"]:
        r -= reward_params["penalty"] * (reward_params["service_target"] - fill_rate)

    return r
print('奖励函数结束')

#值迭代
gamma = 0.95
max_iter = 200#迭代最大次数
edge= 1e-4

V = np.zeros(num_states)
policy = np.zeros(num_states, dtype=int)

for _ in range(max_iter):
    diff = 0
    for s in range(num_states):
        v0 = V[s]
        best_v = -1e18
        best_a = None#初始化

        for a in A[s]:
            future = sum(P.get(s, {}).get(s2, 0) * V[s2] for s2 in P.get(s, {}))#算未来收益
            q = calc_reward(s, a) + gamma * future#贝尔曼=今天的收益+未来收益
            if q > best_v:
                best_v = q
                best_a = a#贪心算法结构辅助

        V[s] = best_v#最佳收益（价值）
        policy[s] = best_a#最佳策略
        diff = max(diff, abs(v0 - V[s]))#取最大值（最优价值，最优动作）

    if diff < edge:
        break

policy_df = state_df.copy()
policy_df["deploy_mdp"] = policy
policy_df.to_csv("mdp_policy_vi.csv", index=False)
print('值迭代完成')
```



*#qlearning* ( 尝试优化 )

**import** random

**Q** = {**s**: {**a**: 0.0 **for** **a** **in** **A**[**s**]} **for** **s** **in** **range**(num\_states)} #原始创建 *q table*

alpha = 0.1

gamma = 0.95

epsilon = 0.2 #经典  $\epsilon$ -greedy, 没用衰减  $\epsilon$ -greedy

episodes = 50

**df\_rl** = **df**.sort\_values("date\_str").reset\_index(drop=True)

**def** choose\_action(**s**):

**if** random.random() < epsilon:

**return** random.choice(**A**[**s**])

**return** max(**Q**[**s**], key=**Q**[**s**].get)

**for** \_ **in** **range**(episodes):

**for** **i** **in** **range**(len(**df\_rl**) - 1):

**s** = **df\_rl**.loc[**i**, "state\_id"]

**s2** = **df\_rl**.loc[**i** + 1, "state\_id"]

**a** = choose\_action(**s**)

**r** = calc\_reward(**s**, **a**)

**Q**[**s**][**a**] += alpha \* (**r** + gamma \* max(**Q**[**s2**].values()) - **Q**[**s**][**a**])

**rl\_policy** = {**s**: max(**Q**[**s**], key=**Q**[**s**].get) **for** **s** **in** **Q**}

**rl\_df** = **state\_df**.copy()

**rl\_df**["deploy\_rl"] = **rl\_policy**.values()

**rl\_df**.to\_csv("mdp\_policy\_qlearning.csv", index=False)

# 6. 最优策略映射到具体日期

**df\_policy** = **df**.copy()

**df\_policy**["optimal\_deploy"] = **df\_policy**["state\_id"].apply(**lambda** **s**: policy[**s**])

**df\_policy**["base\_deploy"] = **df\_policy**["state\_id"].apply(**lambda** **s**: base\_deploy[**s**])

**df\_policy**["adjustment"] = **df\_policy**["optimal\_deploy"] - **df\_policy**["base\_deploy"]

**print**('qlearning\_完成')

```

def action_label(diff):
    if diff < -200:
        return "大幅减少投放"
    elif diff < -50:
        return "减少投放"
    elif abs(diff) <= 50:
        return "维持现状"
    elif diff <= 200:
        return "增加投放"
    else:
        return "大幅增加投放"

df_policy["recommended_action"] = df_policy["adjustment"].apply(action_label)
df_policy["state_value"] = df_policy["state_id"].apply(lambda s: round(V[s], 3))

final_cols = [
    "date_str",
    "season", "mnth", "weekday", "workingday",
    "weather_simplified",
    "demand_level",
    "temp_level", "hum_level",
    "base_deploy",
    "optimal_deploy",
    "adjustment",
    "recommended_action",
    "state_value"
]

final_policy_df = (
    df_policy[final_cols]
    .drop_duplicates()
    .sort_values("date_str")
)

final_policy_df.to_csv(
    "mdp_optimal_policy_by_date.csv",
    index=False,
    encoding="utf-8-sig"
)

print('代码已全部运行成功')

```